

Mobile Tech Lead – Practical Assessment

Candidate: _____ Date: _____

Overview

This assessment has 2 parts:

Part	App	Backend
Day 1	Mini Merchant POS (Offline POS + RBAC)	Provided
Day 2	Doctor Live Consultation (Video Streaming)	Built from scratch

Day 1 – Mini Merchant POS App

A **single app** that combines an Offline POS system with Role-Based Access Control.

Backend URL: <https://pos-test.equilym.com/>

Swagger UI: <https://pos-test.equilym.com/docs>

Do NOT build the backend. Use the provided backend API.

Features to Build

1. Login Screen

- Username and password fields
- Three roles: Admin, Merchant, Employee
- Store JWT token securely after login

2. Product List

- Fetch and display all products from the backend
- Show: name, price, stock
- Pull-to-refresh support

3. Cart

- Add products to cart (select product → enter quantity)
- View cart with all items and running total
- Remove items from cart
- Clear cart

4. Create Order

- Convert cart to order
- Send `localOrderId` (UUID) with every order – this is the **idempotency key**
- Record payment mode: Cash, Card, or Offline

5. Payment Simulation

- Simulate payment after order is created
- Show payment status: `SUCCESS` , `PENDING` , `FAILED`
- Record `paymentRef` (transaction ID)

6. Offline Mode

- When internet is off, save order to local database (Room/SQLite/Hive)
- Order status shown as `PENDING`
- When internet returns, auto-sync order to backend
- Prevent duplicate orders using `localOrderId`
- Show order sync status: `PENDING` → `PAID` → `SYNCED` → `FAILED`

7. Role-Based Access Control

Action	Admin	Merchant	Employee
View products	✓	✓	✓
Add/Edit/Delete products	✓	✗	✗
Add to cart	✓	✓	✓
Create order	✓	✓	✗
Make payment	✓	✓	✗
View own orders	✓	✓	✗
View all orders	✓	✗	✗
Hide restricted buttons	✓	✓	✓

8. Order Status Tracking

- Display status badge on each order: `PENDING` , `PAID` , `SYNCED` , `FAILED`

- Auto-update status after sync completes

9. Reports (Admin only)

- Today's sales: total orders count + total revenue
- Pending sync: count of orders waiting to sync

Backend Details

Base URL: <https://pos-test.equilym.com/api>

Swagger UI: <https://pos-test.equilym.com/docs>

Dummy Users:

Username	Password	Role
admin	admin123	ADMIN
merchant	merchant123	MERCHANT
employee	employee123	EMPLOYEE

Read the full API documentation at: <https://pos-test.equilym.com/docs>

Technical Requirements

- Local database (Room/SQLite/Hive) for offline order storage
 - Auto-sync when internet is available
 - `localOrderId` as idempotency key to prevent duplicate orders
 - Role enforcement on both UI and API
 - Retry queue for failed sync operations
 - Order status tracking: `PENDING` → `PAID` → `SYNCED` or `FAILED`
 - Network connectivity detection (airplane mode toggle)
 - Clean architecture (data/domain/presentation layers)
 - Repository pattern for data access
 - Proper error handling with user feedback
-

Day 2 – Doctor Live Consultation App

Build **everything from scratch** – no backend is provided. Design the architecture, pick the tools, and implement the full app.

Features to Build

1. Login

- Login as Doctor or Patient
- Name/ID entry
- No backend required – use Firebase Auth or mock auth

2. Doctor: Create Room

- Doctor enters room name/ID and creates a video room
- Display shareable room link/ID for patient to join

3. Patient: Join Room

- Patient enters room ID shared by doctor
- Join existing room

4. Video/Audio Call

- Use **Agora SDK** or **WebRTC** (use demo/developer tier)
- Show local video preview
- Show remote participant video
- Audio and video enabled by default

5. Call Controls

- Mute/unmute microphone
- Enable/disable camera
- End call button
- Toggle controls visible on screen

6. Network Handling

- Detect when network drops during call
- Show reconnecting message
- Auto-reconnect when network returns
- Show connection quality indicator

7. Call Duration Timer

- Display elapsed call time (MM:SS format)
- Timer starts when both participants are connected

8. End Call

- Either party can end the call
 - Return to home screen after ending
-

Technical Requirements

- agora SDK or WebRTC properly integrated
 - Room/channel concept implemented correctly
 - Token generation if using Agora
 - Camera and microphone permissions handled
 - Network reconnect logic implemented
 - Call duration timer working
 - Clean architecture
 - Separation of UI and video logic
 - Proper state management
-

General Instructions

1. Read the Swagger docs at <https://pos-test.equilym.com/docs> before calling any API.
2. Do NOT modify the provided backend.
3. Use any framework – Flutter or Android/Kotlin.
4. Internet may go down anytime. The app must work offline.
5. Orders must not disappear after closing the app.
6. Duplicate payments must never be processed twice.
7. Role permissions must work correctly on both UI and API.
8. Clean architecture is expected for both apps.
9. Proper error handling is expected.